

```
#!/usr/bin/env python3
"""
MamaGo Telegram Bot
Бот для приёма заявок с лендинга и оплаты через Kaspi QR
"""

import logging
import json
import os
from datetime import datetime
from telegram import (
    Update, InlineKeyboardButton, InlineKeyboardMarkup,
    ReplyKeyboardMarkup, KeyboardButton
)
from telegram.ext import (
    Application, CommandHandler, MessageHandler,
    CallbackQueryHandler, ConversationHandler, filters,
    ContextTypes
)
from telegram.constants import ParseMode

# — НАСТРОЙКИ —
BOT_TOKEN = "8772939585:AAG4izepyO-4H1Kzk_npbagFhO80AXE4xLs"
ADMIN_CHAT_ID = 474705465
KASPI_QR_URL = "https://pay.kaspi.kz/pay/rrpncedw"
DB_FILE = "applications.json"

# — ТАРИФЫ —
PLANS = {
    "single": {
        "name": "🚗 Разовая поездка",
        "price": "3 000 ₸",
        "price_int": 3000,
        "badge": "",
        "desc": "до 8 км (например, от Хан Шатыра до Mega Silk Way)",
        "features": [
            "Детское кресло",
            "Профессиональный водитель",
            "Подача 7-10 мин",
            "Чистый салон"
        ]
    },
    "priority": {
        "name": "⭐ Mama Priority",
    }
}
```

```

        "price": "45 000 ₮",
        "price_int": 45000,
        "badge": "🔥 ПОПУЛЯРНЫЙ",
        "desc": "в месяц · примерно 16 поездок в месяц",
        "features": [
            "Всё из «Разовой поездки»",
            "Приоритетная подача",
            "Профессиональный водитель",
            "Скидка 15% на поездки",
            "Бесплатная отмена"
        ]
    },
    "family": {
        "name": "👨👩👧 Семейный",
        "price": "60 000 ₮",
        "price_int": 60000,
        "badge": "",
        "desc": "в месяц · до 24 поездок в месяц",
        "features": [
            "Всё из «Мама Priority»",
            "2+ детских кресла",
            "Минивэн по запросу",
            "Скидка 25% на поездки",
            "Персональный менеджер"
        ]
    }
}

```

```

# — СОСТОЯНИЯ ДИАЛОГА —————
(ASK_NAME, ASK_PHONE, ASK_PLAN, CONFIRM_ORDER, PAYMENT_CONFIRM) = range(5)

```

```

# — ЛОГИРОВАНИЕ —————
logging.basicConfig(
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    level=logging.INFO,
    handlers=[
        logging.StreamHandler(),
        logging.FileHandler("bot.log", encoding="utf-8")
    ]
)
logger = logging.getLogger(__name__)

```

```

# — БАЗА ДАННЫХ (JSON) —————
def load_db() -> dict:
    if os.path.exists(DB_FILE):
        with open(DB_FILE, "r", encoding="utf-8") as f:
            return json.load(f)

```

```

    return {"applications": [], "counter": 0}

def save_db(data: dict):
    with open(DB_FILE, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def add_application(app: dict) -> int:
    db = load_db()
    db["counter"] += 1
    app["id"] = db["counter"]
    app["created_at"] = datetime.now().strftime("%d.%m.%Y %H:%M")
    app["status"] = "new"
    db["applications"].append(app)
    save_db(db)
    return app["id"]

def get_applications(status: str = None) -> list:
    db = load_db()
    apps = db["applications"]
    if status:
        apps = [a for a in apps if a.get("status") == status]
    return apps

def update_application_status(app_id: int, status: str):
    db = load_db()
    for app in db["applications"]:
        if app["id"] == app_id:
            app["status"] = status
            break
    save_db(db)

def delete_application(app_id: int) -> bool:
    db = load_db()
    before = len(db["applications"])
    db["applications"] = [a for a in db["applications"] if a["id"] != app_id]
    save_db(db)
    return len(db["applications"]) < before

def is_admin(user_id: int) -> bool:
    return user_id == ADMIN_CHAT_ID

# ——— КЛАВИАТУРЫ —————
def main_keyboard():
    return ReplyKeyboardMarkup(
        [[KeyboardButton("📝 Оставить заявку")],
         [KeyboardButton("💰 Тарифы"), KeyboardButton("❓ О сервисе)]]],
        resize_keyboard=True

```

```

)

def admin_keyboard():
    return ReplyKeyboardMarkup(
        [[KeyboardButton("📁 Все заявки"), KeyboardButton("NEW Новые заявки)],
         [KeyboardButton("✅ Оплаченные"), KeyboardButton("📊 Статистика")]],
        resize_keyboard=True
    )

def plan_keyboard():
    buttons = []
    for key, plan in PLANS.items():
        label = f"{plan['name']} - {plan['price']}"
        if plan["badge"]:
            label = f"{plan['badge']} {plan['name']} - {plan['price']}"
        buttons.append([InlineKeyboardButton(label, callback_data=f"plan_{key}")])
    return InlineKeyboardMarkup(buttons)

def phone_keyboard():
    return ReplyKeyboardMarkup(
        [[KeyboardButton("📞 Отправить номер телефона", request_contact=True)]],
        resize_keyboard=True, one_time_keyboard=True
    )

def payment_keyboard():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("💳 Оплатить через Kaspi QR", url=KASPI_QR_URL)],
        [InlineKeyboardButton("✅ Я оплатил(а)", callback_data="paid")],
        [InlineKeyboardButton("❌ Отменить заявку", callback_data="cancel_order")]
    ])

def app_actions_keyboard(app_id: int, status: str):
    buttons = []
    if status != "paid":
        buttons.append([InlineKeyboardButton("✅ Отметить оплаченным", callback_data=f"paid_{app_id}")])
    if status != "cancelled":
        buttons.append([InlineKeyboardButton("❌ Отменить", callback_data=f"cancel_{app_id}")])
    buttons.append([InlineKeyboardButton("🗑 Удалить заявку", callback_data=f"delete_{app_id}")])
    return InlineKeyboardMarkup(buttons)

# — ВСПОМОГАТЕЛЬНЫЕ —
STATUS_EMOJI = {"new": "NEW", "payment_sent": "⌚", "paid": "✅", "cancelled": "❌"}
STATUS_TEXT = {"new": "Новая", "payment_sent": "Ожидает подтверждения", "paid": "Оплачено", "cancelled": "Отменено"}

def format_application(app: dict) -> str:
    plan = PLANS.get(app.get("plan", ""), {})
    status = app.get("status", "new")

```

```

return (
    f"{'-'*30}\n"
    f"📄 Заявка #{app['id']}\n"
    f"👤 Имя: {app['name']}\n"
    f"☎ Телефон: {app['phone']}\n"
    f"📦 Тариф: {plan.get('name', app.get('plan', ''))}\n"
    f"$💰 Сумма: {plan.get('price', '-')}\n"
    f"📅 17 Дата: {app['created_at']}\n"
    f"📄 Статус: {STATUS_EMOJI.get(status, '?')} {STATUS_TEXT.get(status, st
)

# — HANDLERS: ПОЛЬЗОВАТЕЛЬ —————
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user = update.effective_user
    if is_admin(user.id):
        await update.message.reply_text(
            "👋 Привет, Администратор!\n\nВы вошли в панель управления заявками М
            reply_markup=admin_keyboard()
        )
        return ConversationHandler.END

    await update.message.reply_text(
        f"👋 Привет, {user.first_name}!\n\n"
        f"Добро пожаловать в *МамаGo* 🚗👶\n\n"
        f"Безопасные поездки для мам с детьми.\n"
        f"Детское кресло, профессиональный водитель и подача за 7-10 минут.\n\n"
        f"⚠ Важно: мы возим детей *только вместе с родителями* или сопровождающи
        f"Выберите действие:",
        parse_mode=ParseMode.MARKDOWN,
        reply_markup=main_keyboard()
    )
    return ConversationHandler.END

async def show_about(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "🚗 *О сервисе МамаGo*\n\n"
        "МамаGo — безопасные поездки для мам с детьми.\n\n"
        "✅ Детское кресло в каждом автомобиле\n"
        "✅ Профессиональные, проверенные водители\n"
        "✅ Подача за 7-10 минут\n"
        "✅ Чистый ухоженный салон\n"
        "✅ Минивэн по запросу (тариф Семейный)\n\n"
        "⚠ *Важно:* мы возим детей *только вместе с родителями* или сопровождающ
        "Поездки детей без взрослых не осуществляются.\n\n"
        "Готовы? Нажмите *«📄 Оставить заявку»*",
        parse_mode=ParseMode.MARKDOWN,

```

```

        reply_markup=main_keyboard()
    )

async def show_plans(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = "💰 *Наши тарифы:*\n\n"
    for key, plan in PLANS.items():
        if plan["badge"]:
            text += f"_{plan['badge']}_\n"
            text += f"*{plan['name']}* – *{plan['price']}*\n"
            text += f"_{plan['desc']}_\n"
            for feat in plan['features']:
                text += f"✅ {feat}\n"
            text += "\n"
    text += (
        "⚠️ Важно: мы возим детей *только вместе с родителями* или сопровождающим  

        📝 Для записи нажмите *«✍️ Оставить заявку»*"
    )
    await update.message.reply_text(text, parse_mode=ParseMode.MARKDOWN, reply_ma

async def start_application(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data.clear()
    await update.message.reply_text(
        "✍️ *Оформление заявки*\n\nШаг 1 из 3\n\nКак вас зовут? Введите имя и фамилию\n\n",
        parse_mode=ParseMode.MARKDOWN,
        reply_markup=ReplyKeyboardMarkup([["❌ Отмена"]], resize_keyboard=True)
    )
    return ASK_NAME

async def got_name(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text.strip()
    if text == "❌ Отмена":
        await update.message.reply_text("Заявка отменена.", reply_markup=main_keyboard())
        return ConversationHandler.END
    if len(text) < 2:
        await update.message.reply_text("Пожалуйста, введите ваше имя (минимум 2 символа)\n\n",
        parse_mode=ParseMode.MARKDOWN,
        reply_markup=main_keyboard())
        return ASK_NAME
    context.user_data["name"] = text
    await update.message.reply_text(
        f"✅ Отлично, {text}!\n\nШаг 2 из 3\n\n",
        parse_mode=ParseMode.MARKDOWN,
        reply_markup=phone_keyboard()
    )
    return ASK_PHONE

```

```

async def got_phone_contact(update: Update, context: ContextTypes.DEFAULT_TYPE):
    phone = update.message.contact.phone_number
    if not phone.startswith("+"):
        phone = "+" + phone
    context.user_data["phone"] = phone
    return await _show_plan_selection(update, context)

async def got_phone_text(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text.strip()
    if text == "❌ Отмена":
        await update.message.reply_text("Заявка отменена.", reply_markup=main_key)
        return ConversationHandler.END
    phone = text.replace(" ", "").replace("-", "")
    if not (phone.startswith("+7") or phone.startswith("8")) or len(phone) < 11:
        await update.message.reply_text("❗ Введите корректный номер (например, +
        return ASK_PHONE
    context.user_data["phone"] = phone
    return await _show_plan_selection(update, context)

async def _show_plan_selection(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Шаг 3 из 3\n\n📦 Выберите тариф:", reply_mar
    return ASK_PLAN

async def got_plan(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    plan_key = query.data.replace("plan_", "")
    plan = PLANS.get(plan_key)
    if not plan:
        return ASK_PLAN
    context.user_data["plan"] = plan_key
    text = (
        f"📋 *Проверьте заявку:*\n\n"
        f"👤 Имя: {context.user_data['name']}\n"
        f"📞 Телефон: {context.user_data['phone']}\n"
        f"📦 Тариф: {plan['name']}\n"
        f"💰 Стоимость: {plan['price']}\n"
        f"_{plan['desc']}_\n\nВсё верно?"
    )
    await query.edit_message_text(text, parse_mode=ParseMode.MARKDOWN, reply_mark
        [InlineKeyboardButton("✅ Подтвердить", callback_data="confirm_yes")],
        [InlineKeyboardButton("✎ Изменить тариф", callback_data="confirm_no")]

```

```

)))
return CONFIRM_ORDER

```

```

async def confirm_order(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    if query.data == "confirm_no":
        await query.edit_message_text("Выберите тариф:", reply_markup=plan_keyboa
        return ASK_PLAN

```

```

app = {
    "user_id": update.effective_user.id,
    "name": context.user_data["name"],
    "phone": context.user_data["phone"],
    "plan": context.user_data["plan"],
}

```

```

app_id = add_application(app)
plan = PLANS[context.user_data["plan"]]

```

```

try:
    await context.bot.send_message(
        chat_id=ADMIN_CHAT_ID,
        text=(
            f"🔔 *НОВАЯ ЗАЯВКА!* \n\n"
            f"📌 Заявка #{app_id} \n"
            f"👤 Имя: {app['name']} \n"
            f"📞 Телефон: {app['phone']} \n"
            f"📦 Тариф: {plan['name']} \n"
            f"💰 Сумма: {plan['price']} \n"
            f"📄 Статус:  Новая"
        ),
        parse_mode=ParseMode.MARKDOWN,
        reply_markup=app_actions_keyboard(app_id, "new")
    )

```

```

except Exception as e:
    logger.error(f"Не удалось уведомить администратора: {e}")

```

```

payment_note = (
    f"Введите сумму: {plan['price']}"
    if plan["price_int"] == 3000
    else f"Введите сумму: {plan['price']} (оплата за месяц)"
)

```

```

await query.edit_message_text(
    f"🎉 *Заявка #{app_id} принята!* \n\n"
    f"📦 Тариф: {plan['name']} \n"
    f"💰 К оплате: {plan['price']} \n\n"

```



```

f"_____\\n"
f"👤 *Оплата через Kaspi QR*\\n\\n"
f"1 ⌨️ Нажмите кнопку «Оплатить через Kaspi QR»\\n"
f"2 📱 Отсканируйте QR-код в приложении Kaspi\\n"
f"3 {payment_note}\\n"
f"4 🔄 Вернитесь сюда и нажмите «✅ Я оплатил(а)»\\n\\n"
f"После подтверждения мы свяжемся с вами 🚗",
parse_mode=ParseMode.MARKDOWN,
reply_markup=payment_keyboard()
)
return PAYMENT_CONFIRM

```

```

async def payment_done(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    if query.data == "cancel_order":
        await query.edit_message_text("❌ Заявка отменена. Ждём вас! 🚗")
        await query.message.reply_text("Главное меню:", reply_markup=main_keyboard)
        return ConversationHandler.END

    _update_status_by_user(update.effective_user.id, "payment_sent")
    await query.edit_message_text(
        "⌚ *Спасибо! Оплата на проверке.*\\n\\n"
        "Наш менеджер проверит платёж и свяжется с вами в ближайшее время. 💕",
        parse_mode=ParseMode.MARKDOWN
    )
    await query.message.reply_text("Главное меню:", reply_markup=main_keyboard())
    try:
        await context.bot.send_message(
            chat_id=ADMIN_CHAT_ID,
            text=(
                f"⌚ *Пользователь сообщил об оплате!*\\n\\n"
                f"👤 {query.from_user.first_name} (@{query.from_user.username or ''})"
                f"Проверьте поступление средств в Kaspi."
            ),
            parse_mode=ParseMode.MARKDOWN
        )
    except Exception as e:
        logger.error(e)
    return ConversationHandler.END

```

```

def _update_status_by_user(user_id: int, status: str):
    db = load_db()
    for app in reversed(db["applications"]):
        if app.get("user_id") == user_id and app.get("status") in ("new", "paymen

```

```

        app["status"] = status
        break
    save_db(db)

async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Отменено.", reply_markup=main_keyboard())
    return ConversationHandler.END

# — HANDLERS: АДМИНИСТРАТОР —————
async def admin_all_apps(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        return
    apps = get_applications()
    if not apps:
        await update.message.reply_text("🚫 Заявок пока нет.", reply_markup=admin_keyboard())
        return
    await update.message.reply_text(f"📁 Все заявки ({len(apps)}):", reply_markup=admin_keyboard())
    for app in apps[-20:]:
        await update.message.reply_text(format_application(app),
                                         reply_markup=app_actions_keyboard(app["id"]))

async def admin_new_apps(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        return
    apps = get_applications(status="new")
    if not apps:
        await update.message.reply_text("✅ Новых заявок нет.", reply_markup=admin_keyboard())
        return
    await update.message.reply_text(f"🆕 Новые заявки ({len(apps)}):", reply_markup=admin_keyboard())
    for app in apps:
        await update.message.reply_text(format_application(app),
                                         reply_markup=app_actions_keyboard(app["id"]))

async def admin_paid_apps(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        return
    apps = get_applications(status="paid")
    if not apps:
        await update.message.reply_text("Оплаченных заявок нет.", reply_markup=admin_keyboard())
        return
    await update.message.reply_text(f"✅ Оплаченные ({len(apps)}):", reply_markup=admin_keyboard())
    for app in apps:
        await update.message.reply_text(format_application(app),
                                         reply_markup=app_actions_keyboard(app["id"]))

```

```

async def admin_stats(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        return
    db = load_db()
    apps = db["applications"]
    counts = {s: sum(1 for a in apps if a.get("status") == s)
               for s in ("new", "payment_sent", "paid", "cancelled")}
    revenue = sum(PLANS.get(a.get("plan", ""), {}).get("price_int", 0)
                  for a in apps if a.get("status") == "paid")
    await update.message.reply_text(
        f" *Статистика MamaGo*\n\n"
        f" Всего заявок: {len(apps)}\n"
        f" Новые: {counts['new']}\n"
        f" Ожидают подтверждения: {counts['payment_sent']}\n"
        f" Оплачены: {counts['paid']}\n"
        f" Отменены: {counts['cancelled']}\n\n"
        f" Выручка (оплаченные): {revenue:,} ₸".replace(",", " "),
        parse_mode=ParseMode.MARKDOWN,
        reply_markup=admin_keyboard()
    )

```

```

async def admin_callback(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    if not is_admin(query.from_user.id):
        await query.answer("Нет доступа")
        return
    await query.answer()
    data = query.data
    if data.startswith("mark_paid_"):
        app_id = int(data.replace("mark_paid_", ""))
        update_application_status(app_id, "paid")
        await query.edit_message_text(query.message.text + "\n\n Отмечено как с
                                   reply_markup=app_actions_keyboard(app_id, "
    elif data.startswith("cancel_app_"):
        app_id = int(data.replace("cancel_app_", ""))
        update_application_status(app_id, "cancelled")
        await query.edit_message_text(query.message.text + "\n\n Заявка отменен
                                   reply_markup=app_actions_keyboard(app_id, "
    elif data.startswith("delete_app_"):
        app_id = int(data.replace("delete_app_", ""))
        if delete_application(app_id):
            await query.edit_message_text(f" Заявка #{app_id} удалена.")
        else:
            await query.answer("Заявка не найдена")

```

```

# — ЗАПУСК —————
def main():
    application = Application.builder().token(BOT_TOKEN).build()

    conv = ConversationHandler(
        entry_points=[MessageHandler(filters.Regex("^📝 Оставить заявку$"), start
        states={
            ASK_NAME: [MessageHandler(filters.TEXT & ~filters.COMMAND, got_
            ASK_PHONE: [MessageHandler(filters.CONTACT, got_phone_contact),
                        MessageHandler(filters.TEXT & ~filters.COMMAND, got_
            ASK_PLAN: [CallbackQueryHandler(got_plan, pattern="^plan_")],
            CONFIRM_ORDER: [CallbackQueryHandler(confirm_order, pattern="^confir
            PAYMENT_CONFIRM: [CallbackQueryHandler(payment_done, pattern="^(paid|c
        },
        fallbacks=[CommandHandler("cancel", cancel)],
        allow_reentry=True
    )

    application.add_handler(conv)
    application.add_handler(CommandHandler("start", start))
    application.add_handler(MessageHandler(filters.Regex("^? О сервисе$"), show_
    application.add_handler(MessageHandler(filters.Regex("^💰 Тарифы$"), show_pla
    application.add_handler(MessageHandler(filters.Regex("^📁 Все заявки$"), admi
    application.add_handler(MessageHandler(filters.Regex("^NEW Новые заявки$"), ad
    application.add_handler(MessageHandler(filters.Regex("^✅ Оплаченные$"), admi
    application.add_handler(MessageHandler(filters.Regex("^🇮🇹 Статистика$"), admi
    application.add_handler(CallbackQueryHandler(admin_callback, pattern="^(mark_

    logger.info("✅ MamaGo бот запущен!")
    application.run_polling(drop_pending_updates=True)

if __name__ == "__main__":
    main()

```